

SafeUpload

Documento de Arquitetura de Software

Versão 2.0

Campo	Informação
Grupo	Prevenção de vazamento de dados
Integrantes	Victor Nogueira da Nova Bonato, Pedro Campos Canafístula, Luiz Henrique Alves Rodrigues e Lucas Ferreira Coelho
Instituição	Universidade Católica de Brasília
Disciplina	Análise e Projeto de Software
Data	18/06/2026

Histórico da Revisão

Data	Versão	Descrição	Autor
10/06/2026	1.0	Estruturação inicial do documento e preenchimento das seções básicas do Documento de Arquitetura de Software.	Grupo SafeUpload
12/06/2026	1.5	Refinamento do conteúdo técnico, revisão dos diagramas iniciais e alinhamento com o MVP.	Grupo SafeUpload
18/06/2026	2.0	Revisão final conforme o template da disciplina, padronização da estrutura, ajuste da arquitetura centralizada, revisão da coerência com os requisitos, melhoria dos espaçamentos e atualização das referências.	Grupo SafeUpload

Índice Analítico

Seção	Página
1. Introdução	4
1.1 Finalidade	4
1.2 Escopo	4
1.3 Definições, Acrônimos e Abreviações	5
2. Restrições e requisitos arquiteturais	6
3. Visão de Casos de Uso	7
4. Visão Lógica	8
4.1 Representação do domínio da aplicação	8
4.2 Decisões arquiteturais	9
4.3 Representação da arquitetura lógica	10
4.4 Representação do funcionamento da arquitetura	11
5. Visão de Processos	11
6. Visão da Implementação	12
7. Visão de Implantação	12
8. Visão de Dados	13
9. Volume e Desempenho	14
10. Referências	14

Documento de Arquitetura de Software

1. Introdução

O SafeUpload é um protótipo acadêmico centralizado voltado à prevenção de vazamento acidental de dados em arquivos digitais. A aplicação permite que o usuário acesse uma interface web, envie um arquivo para inspeção e consulte o resultado antes de decidir se continuará o envio por outro canal.

No MVP, o termo upload significa apenas envio do arquivo ao SafeUpload para análise. O sistema não envia automaticamente arquivos para e-mail, nuvem ou serviços externos. Essa delimitação foi mantida para deixar a arquitetura simples, verificável e adequada ao contexto da disciplina.

1.1 Finalidade

Este documento registra as principais decisões arquiteturais do SafeUpload e apresenta as visões necessárias para orientar a implementação, a revisão acadêmica e a manutenção do MVP. A estrutura segue o template fornecido na disciplina e utiliza o modelo de visões 4+1 como referência para organizar a arquitetura.

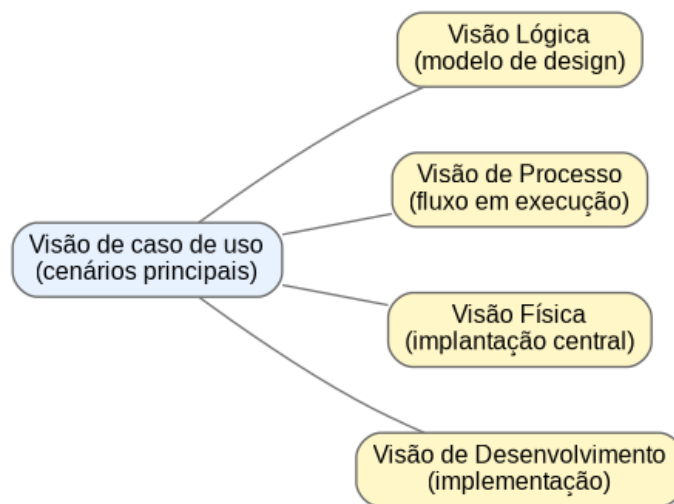


Figura 1 - Visões 4+1 da arquitetura de software, adaptado de Kruchten (1995).

1.2 Escopo

A arquitetura descrita cobre o MVP 1.0 do SafeUpload, documentado nesta versão 2.0 do Documento de Arquitetura. O sistema funciona como uma aplicação web centralizada e oferece o fluxo principal de inspeção de arquivos, extraíndo texto de formatos suportados, aplicando regras de detecção e classificando o resultado como aprovado, bloqueado ou rejeitado.

Integrações automáticas com serviços externos, OCR, análise de imagens e operação corporativa em larga escala não fazem parte do MVP. Esses pontos aparecem apenas como limitações ou possibilidades de evolução.

1.3 Definições, Acrônimos e Abreviações

<i>Termo</i>	<i>Definição</i>
<i>DLP</i>	Data Loss Prevention. Conjunto de práticas e mecanismos usados para reduzir o risco de vazamento de informações.
<i>LGPD</i>	Lei Geral de Proteção de Dados Pessoais - Lei nº 13.709/2018.
<i>MVP</i>	Produto Mínimo Viável usado para validar o núcleo da proposta.
<i>Upload para inspeção</i>	Envio de arquivo ao SafeUpload, por interface web, para inspeção no servidor central da aplicação. Não implica envio automático para terceiros.
<i>Allowlist</i>	Lista controlada de exceções utilizada para tratar falsos positivos recorrentes. No MVP, é tratada como evolução planejada ou apoio experimental.
<i>UML</i>	Unified Modeling Language. Linguagem de modelagem utilizada para representar casos de uso, classes, pacotes, sequência e implantação.

2. Restrições e requisitos arquiteturais

A arquitetura foi definida com base nos requisitos funcionais e não funcionais levantados para o SafeUpload. A tabela a seguir resume os atributos de qualidade mais relevantes e a solução adotada para cada um deles, seguindo a lógica proposta no template da disciplina.

<i>Atributo de qualidade</i>	<i>Requisito de arquitetura</i>	<i>Solução</i>
Desempenho	Analisar arquivos de até 10 MB em até 5 segundos no ambiente de referência.	Limite máximo de 20 MB por arquivo, leitura controlada pela API central e extratores específicos por formato.
Interoperabilidade	Processar conteúdo textual extraível de .txt, .pdf, .docx, .csv e .xlsx.	Uso de bibliotecas executadas no servidor para cada formato. OCR e leitura de texto em imagem ficam fora do MVP.
Usabilidade	Apresentar resposta clara para usuário final e administrador.	Interface web acessada por navegador, com estados de aprovado, bloqueado ou rejeitado, incluindo motivo do bloqueio sem expor valores reais.
Confiabilidade	Evitar falhas por formato inválido, arquivo grande ou conteúdo não processável.	Validação de extensão, tamanho e tratamento de exceções no servidor antes de executar a inspeção completa. O conteúdo temporário também deve ser descartado em caso de erro.
Segurança	Não armazenar conteúdo integral dos arquivos e restringir funções administrativas.	Conteúdo temporário em memória no servidor, logs com dados mascarados, hash de senha, sessão da aplicação, CSRF, separação por perfil e recomendação de HTTPS em ambiente centralizado.
Facilidade de manutenção	Permitir alteração das regras sem refazer toda a aplicação.	Separação em camadas: apresentação, aplicação, domínio e infraestrutura.
Portabilidade	Permitir acesso por navegadores modernos e execução em ambiente de servidor compatível.	Aplicação Python 3.11+ executada em servidor central e acessada por navegador web.
Escalabilidade	Atender ao uso acadêmico centralizado e de baixa concorrência no MVP.	Um servidor central para demonstração. Para uso corporativo amplo, seria necessário avaliar banco, autenticação, balanceamento e infraestrutura.
Disponibilidade	Manter o fluxo principal disponível enquanto o servidor central estiver ativo.	A disponibilidade fica vinculada ao servidor da aplicação, ao banco de dados e à rede usada pelos usuários.

Tabela 1 - Restrições e requisitos arquiteturais

3. Visão de Casos de Uso

A visão de casos de uso apresenta as funcionalidades consideradas pela arquitetura. Para manter coerência com o Documento de Requisitos, as histórias HU-01 a HU-05 formam o núcleo obrigatório do MVP, HU-06 e HU-07 são complementares, HU-08 e HU-09 ficam como evolução planejada ou apoio experimental simples, e HU-10 permanece fora do MVP.

ID	História de usuário	Situação
HU-01	Inspecionar arquivo submetido pelo usuário	MVP obrigatório
HU-02	Detectar categorias de risco no conteúdo extraído	MVP obrigatório
HU-03	Bloquear arquivos com ocorrências válidas	MVP obrigatório
HU-04	Registrar eventos em log de auditoria	MVP obrigatório
HU-05	Exibir motivo do bloqueio ao usuário	MVP obrigatório
HU-06	Autenticar usuário e controlar acesso	MVP complementar
HU-07	Configurar categorias de detecção	MVP complementar
HU-08	Gerenciar lista de exceções controlada	Evolução planejada
HU-09	Gerar relatório consolidado	Evolução planejada
HU-10	Integrar inspeção a serviço externo	Fora do MVP

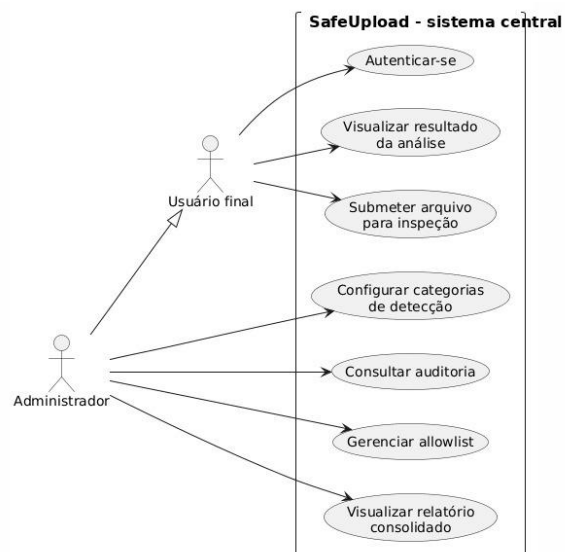


Figura 2 - Diagrama de casos de uso UML do SafeUpload.

Os casos “Gerenciar allowlist” e “Visualizar relatório consolidado” representam evolução planejada ou apoio experimental. Eles não fazem parte do núcleo obrigatório do MVP.

4. Visão Lógica

A visão lógica descreve a organização interna do SafeUpload. Ela mostra o domínio principal da aplicação, as decisões arquiteturais, a divisão em pacotes e o funcionamento das camadas no cenário de inspeção de arquivo.

4.1 Representação do domínio da aplicação

O diagrama de classes da UML representa os conceitos centrais do domínio e seus relacionamentos. No SafeUpload, os principais objetos estão ligados à submissão de arquivos, inspeção, categorias de detecção, ocorrências de risco, auditoria e usuários.

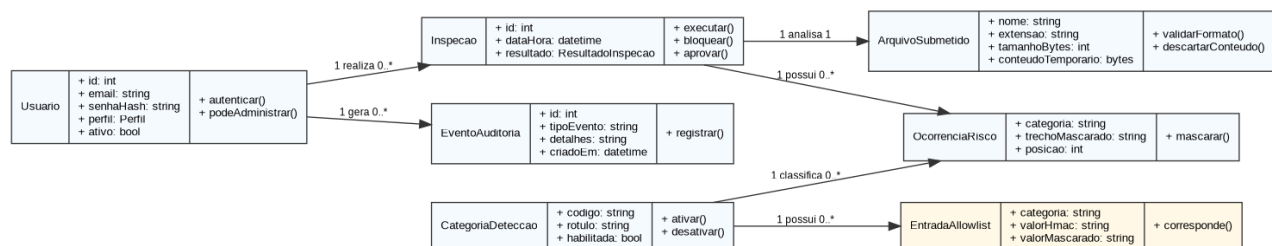


Figura 3 - Diagrama de classes UML do domínio da aplicação.

A classe **Inspecao** concentra o resultado da análise e se relaciona com o arquivo submetido e com as ocorrências identificadas. **CategoriaDeteccao** representa as regras habilitadas. **EventoAuditoria** registra a rastreabilidade sem guardar o conteúdo integral do arquivo. **EntradaAllowlist** aparece como estrutura prevista para tratar falsos positivos recorrentes, mas não altera o núcleo obrigatório do MVP.

4.2 Decisões arquiteturais

<i>Item arquitetural</i>	<i>Decisão</i>	<i>Justificativa</i>
Estilo arquitetural	Arquitetura em camadas, com aplicação web centralizada.	A separação entre interface, orquestração, regras de negócio e infraestrutura facilita manutenção, testes e implantação em servidor central.
Linguagem de programação	Python 3.11 ou superior.	A linguagem atende bem ao protótipo e possui bibliotecas prontas para extração textual de arquivos.
Framework adotado	FastAPI, Uvicorn e Jinja2.	FastAPI oferece rotas HTTP para a aplicação central; Uvicorn executa o servidor; Jinja2 renderiza as páginas HTML.
Formas de persistência	Persistência centralizada no servidor da aplicação.	O MVP precisa registrar usuários, categorias e auditoria em um ponto único, sem armazenar o conteúdo integral dos arquivos.
Banco de dados	SQLite centralizado no servidor da aplicação.	Foi adotado pela simplicidade do MVP e pela baixa concorrência esperada. Em uso corporativo, poderia ser substituído por um banco relacional mais robusto.
Serviços de terceiros	Nenhum serviço externo obrigatório.	O fluxo principal depende do servidor SafeUpload e das bibliotecas da aplicação, sem chamadas automáticas a APIs externas.

Tabela 2 - Decisões arquiteturais

4.3 Representação da arquitetura lógica

O diagrama de pacotes da UML mostra a modularização da aplicação. A dependência principal parte da camada de apresentação para a aplicação, que coordena as regras de domínio e usa a infraestrutura para extração e persistência.

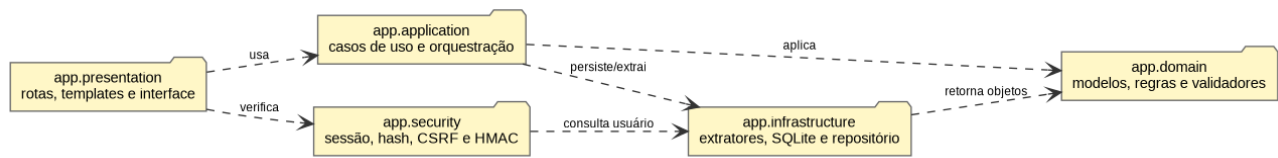


Figura 4 - Diagrama de pacotes UML da arquitetura lógica.

Pacote	Responsabilidade
app.presentation	Rotas HTTP, templates HTML, formulários, páginas de inspeção, login, configuração, auditoria e relatórios acessados pelo navegador.
app.application	Coordenação dos casos de uso, especialmente a inspeção no servidor, o descarte do conteúdo temporário e a gravação da auditoria.
app.domain	Modelos do negócio, validadores de CPF, CNPJ, cartão, indícios de senha, enumerações de resultado e mascaramento.
app.infrastructure	Extratores por formato, repositório, acesso ao banco central e operações técnicas de persistência.
app.security	Controle de sessão, hash de senha, token CSRF e HMAC para valores que não devem ser guardados em texto claro.

4.4 Representação do funcionamento da arquitetura

O diagrama de sequência da UML apresenta o cenário principal de inspeção centralizada. Ele mostra a colaboração entre usuário, interface web, rota da API, serviço de aplicação, extrator, validadores e repositório.

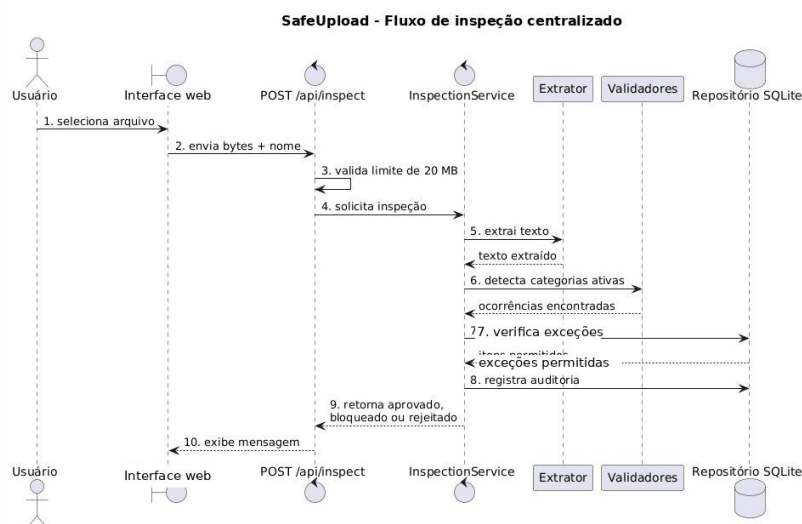


Figura 5 - Diagrama de sequência UML do fluxo de inspeção centralizado.

Ao final do fluxo, o resultado pode ser aprovado, bloqueado ou rejeitado. O conteúdo integral do arquivo é descartado após a análise ou em caso de erro, enquanto a auditoria registra somente metadados, categorias identificadas e trechos mascarados quando necessário.

5. Visão de Processos

No MVP, o SafeUpload executa como uma aplicação centralizada em um servidor da aplicação. O navegador do usuário se comunica com o servidor FastAPI/Uvicorn por HTTP/HTTPS. O fluxo não depende de filas, serviços distribuídos ou processos externos.

- Processo principal: servidor central FastAPI/Uvicorn, responsável por receber requisições, renderizar páginas e executar o fluxo de inspeção.
- Execução da análise: leitura temporária do arquivo no servidor, extração textual, validação das categorias e descarte do conteúdo integral ao final.
- Persistência: gravação de usuários, categorias, auditoria e, quando aplicável, exceções controladas no banco de dados central.
- Comunicação: requisições HTTP/HTTPS entre navegador e servidor central, sem transmissão automática para terceiros.

6. Visão da Implementação

A implementação é organizada em módulos que acompanham a divisão lógica da arquitetura. A tabela a seguir resume os componentes mais importantes do ponto de vista técnico.

Componente	Tecnologia ou função
Servidor central	FastAPI e Uvicorn executados no servidor da aplicação.
Interface	Templates Jinja2, HTML, CSS e JavaScript acessados por navegador web.
Extração de texto	PyMuPDF para PDF, python-docx para DOCX, openpyxl para XLSX e bibliotecas nativas para TXT e CSV.
Persistência	SQLite centralizado no servidor da aplicação, acessado pelo repositório.
Segurança da aplicação	PBKDF2 para hash de senhas, HMAC para valores sensíveis da allowlist, token CSRF, cookie de sessão e recomendação de HTTPS.
Testes	Testes automatizados para validadores, extração, inspeção e repositório. A validação final de desempenho deve ser feita em ambiente representativo.

Tabela 3 - Componentes principais da implementação

7. Visão de Implantação

A implantação do MVP ocorre em um servidor central da aplicação. O usuário acessa a interface pelo navegador, enquanto o processamento, a auditoria e a persistência ficam concentrados no ambiente do servidor.

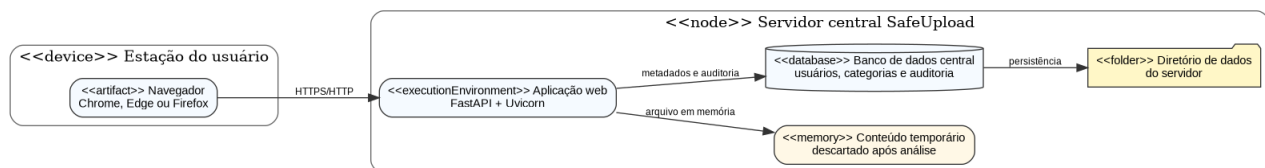


Figura 6 - Diagrama de implantação UML centralizada do SafeUpload.

Como a solução é um MVP acadêmico centralizado, uma base simples no servidor atende à demonstração inicial. Em uma implantação corporativa, seria necessário avaliar autenticação institucional, banco de dados compartilhado, logs centralizados, políticas de backup e controles adicionais de segurança.

8. Visão de Dados

A visão de dados descreve as informações persistidas pela aplicação. O SafeUpload não armazena o conteúdo integral dos arquivos analisados. A persistência fica restrita a autenticação, configuração, auditoria e tratamento controlado de falsos positivos.

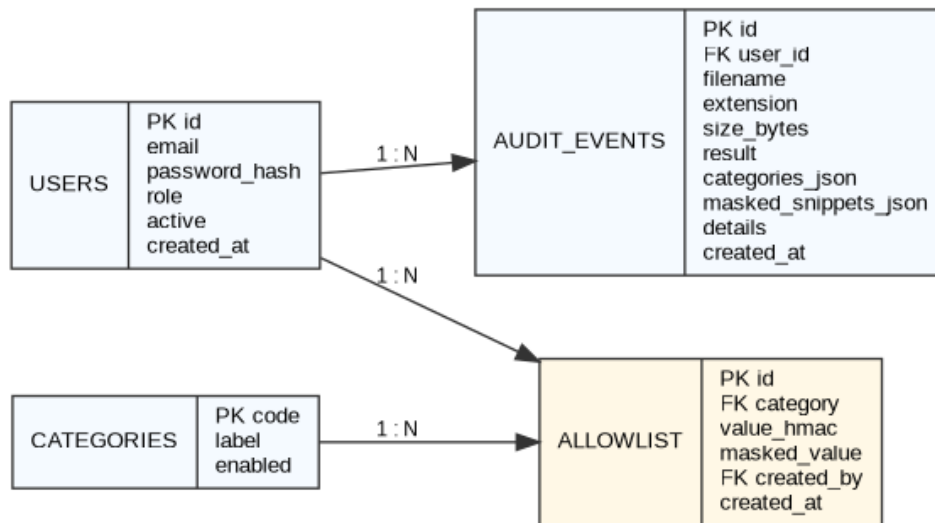


Figura 7 - Modelo Entidade-Relacionamento resumido do banco central do MVP.

Estrutura	Finalidade	Observação
USERS	Autenticação e perfil de acesso.	Senha armazenada como hash; não há senha em texto claro.
CATEGORIES	Categorias de detecção habilitadas ou desabilitadas.	Permite configurar o comportamento das regras sem alterar o código.
AUDIT_EVENTS	Rastreabilidade das inspeções e ações administrativas.	Armazena metadados, resultado, categorias e trechos mascarados, nunca o arquivo integral.
ALLOWLIST	Exceções controladas para falsos positivos recorrentes.	Valor real representado por HMAC e forma mascarada; tratado como evolução planejada/apoio experimental.

Tabela 4 - Estruturas persistidas no banco central

9. Volume e Desempenho

O dimensionamento inicial foi definido para uma aplicação centralizada de uso interativo. Os valores abaixo servem como referência para os testes do MVP e não representam uma implantação corporativa.

Parâmetro	Definição
Usuários simultâneos	Baixa concorrência no MVP acadêmico; estimativa inicial de 5 a 10 usuários em testes controlados.
Tamanho máximo aceito	20 MB por arquivo.
Referência de desempenho	Até 5 segundos para arquivos de até 10 MB em ambiente definido nos testes.
Formatos suportados	.txt, .pdf, .docx, .csv e .xlsx.
Operação	Aplicação central acessada por navegador, em rede interna ou ambiente publicado para testes.
Volume inicial de dados	Baixo volume, composto por usuários, categorias, auditoria e exceções controladas.
Retenção de logs	Não definida para o MVP; deve ser configurada em versão corporativa.
Backup	Não detalhado no MVP; em produção deve seguir rotina do ambiente central.
Disponibilidade	Vinculada ao servidor central, ao banco de dados e à rede de acesso dos usuários.

Tabela 5 - Parâmetros de volume e desempenho

Limitações conhecidas: o SafeUpload não substitui uma plataforma corporativa de DLP, não realiza OCR, não interpreta imagens incorporadas e não garante eliminação absoluta de falsos positivos ou falsos negativos. O resultado aprovado significa apenas que as regras ativas não encontraram ocorrências no conteúdo textual extraível.

10. Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Texto compilado. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709compilado.htm. Acesso em: 10 jun. 2026.

KRUCHTEN, Philippe. Architectural Blueprints: The “4+1” View Model of Software Architecture. IEEE Software, v. 12, n. 6, p. 42-50, 1995. Disponível em: <https://ieeexplore.ieee.org/document/469759>. Acesso em: 12 jun. 2026.

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language (OMG UML), Version 2.5.1. Needham: OMG, 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1/>. Acesso em: 18 jun. 2026.

OPENAI. ChatGPT. Ferramenta utilizada como apoio para polimento, revisão textual e refinamento da organização do documento. A definição do escopo, a validação técnica e a responsabilidade pela versão final permanecem com o grupo SafeUpload, 2026.